

ENTERPRISE MOBILE APPLICATION DEVELOPMENT BOOTCAMP

AdventureWorks Customer Companion

Android Capstone Project Requirements

Team Size 4 Students

Customer focused product discovery and order tracking

Project Overview

AdventureWorks wants to improve its mobile customer experience. Your team will design and build an Android application that helps customers discover products, remember products that interest them, and review their existing orders. The application will use the AdventureWorks REST API for remote data and local Android persistence for customer preferences and browsing history.

This project is intentionally written as a set of business and technical requirements rather than a finished backlog. Your team must analyze the requirements, inspect the API, create and refine user stories, define acceptance criteria, estimate the work, and deliver a working product increment.

Project Snapshot

Item	Requirement
Product	AdventureWorks Customer Companion
Platform	Android using Kotlin and Jetpack Compose
Team	Four students working as one Scrum team
API	AdventureWorks API at api.bootcampcentral.com
Primary areas	Discover, Orders, and Saved
Persistence	Saved products and recently viewed products
Delivery goal	A reliable, accessible MVP with selected enhancements

Business Scenario

AdventureWorks customers currently lack a convenient mobile experience for exploring the product catalog and checking their existing orders. The first release of Customer Companion will provide a focused set of self-service capabilities without attempting to implement a complete e-commerce checkout system.

Primary Customer Journey

Area	Customer journey
Discover	Browse, search, filter, open product details, and evaluate availability
Remember	Save products intentionally and return to recently viewed products
Orders	Review order history, locate an order, and inspect order details and status

Expected Learning Outcomes

- Apply MVVM and Clean Architecture to a multi-feature Android application.
- Use Retrofit, OkHttp, `kotlinx.serialization`, coroutines, and `StateFlow` to retrieve and present remote data.
- Use Room to persist customer preferences and browsing history.
- Apply Hilt to assemble repositories, use cases, `ViewModels`, and data sources.

- Handle loading, empty, failure, timeout, and partial-success states deliberately.
- Work as a Scrum team to refine requirements, divide work, integrate features, test, and demonstrate a product increment.

Application Scope

The required application contains three top-level destinations. Teams may organize secondary screens differently, but the customer must be able to move predictably among the catalog, orders, saved products, and product or order details.

Destination	Required content
Discover	Personalized home content, search and filters, recently viewed products, product catalog, and product details
Orders	Customer order history, order filtering or sorting, and order details
Saved	Persisted saved products with navigation to product details

Demo Customer Context

For this prototype, the instructor will provide each team with an approved demo customer profile containing a customer ID and display name. Authentication, account registration, and production authorization are outside the required scope.

- The application must maintain one active demo customer session.
- Order history and order details must be retrieved within the active customer context.
- The customer identifier must not be duplicated as a magic number across composables or ViewModels.
- The customer's display name should appear in an appropriate location such as the home or order-history screen.
- The interface must clearly present this as a demo customer context rather than a production authentication flow.

Out of Scope

- Shopping cart and checkout
- Payment processing
- Creating or modifying orders
- Customer registration and password recovery
- Administrative employee or inventory management
- Production-grade authentication and authorization
- Full bidirectional offline synchronization

API Discovery

Teams are responsible for examining the Swagger documentation and identifying the API resources, parameters, and response models that support the requirements. Application requirements should drive endpoint selection.

Required Application Features

All sixteen requirements in this section are part of the minimum viable product. Teams may refine and split them into smaller user stories, but may not remove the underlying customer capability without instructor approval.

1 Customer Home

Customers need a useful starting point when they open the application. The home experience should provide clear access to the product catalog and display personalized information such as recently viewed products and a summary of recent order activity.

- A failure loading one section must not unnecessarily hide another usable section.
- Locally available information should remain visible when a remote request fails.

2 Product Catalog

Customers need to browse the AdventureWorks product catalog. Each product summary should contain enough useful information to distinguish it from other products, such as its name, category, price, image, or availability when those values are provided by the API.

3 Product Search

Customers often know all or part of the product name they want. The application must provide a search capability that updates the visible results and communicates clearly when no products match.

- Search should be case-insensitive.
- Clearing the search should restore the unfiltered catalog.

4 Product Filtering and Sorting

Customers need to narrow and organize the catalog. The application must provide at least one meaningful filter and at least one meaningful sort option supported by the available data.

- Examples include category, product type, availability, price, or alphabetical order.
- Search, filtering, and sorting should work together predictably.

Required Application Features Continued

5 Product Details

Customers need to select a product and view customer-relevant information about it. The screen should transform technical API fields into a readable presentation rather than displaying raw field names and values.

- A product-detail failure must provide a meaningful message and recovery action.
- The product is not considered recently viewed until its details load successfully.

6 Customer Friendly Availability

Customers need an understandable indication of whether a product appears to be available. The application must translate inventory information into classifications such as In Stock, Low Stock, Unavailable, or Availability Unknown.

- The interpretation should be centralized rather than duplicated across composables.
- Availability must not be communicated through color alone.

7 Saved Products

Customers need to deliberately save products they may want to revisit later. Saved products must persist after the application is closed and reopened.

- Customers can save and remove individual products.
- The Saved destination displays all saved products and supports navigation to product details.
- Removing a saved product must not remove it from recently viewed history.

8 Recently Viewed Products

When product details load successfully, the application must record the product in recently viewed history. This history must remain available after the application is closed and reopened.

- Products appear in most-recent-first order.
- The same product must not appear more than once.
- Viewing an existing product again moves it to the first position.
- Only the ten most recently viewed products are retained.
- Customers can clear the history without affecting saved products.
- Selecting a recent product opens its details.

Required Application Features Continued

9 Order History

Customers need to review orders associated with the active demo customer. Each order summary should display enough information to distinguish it from other orders, such as order number, date, status, and total amount when available.

- The application must provide an appropriate empty state when no orders are available.
- Money values must be represented and formatted without floating-point rounding errors.

10 Order Filtering or Sorting

Customers need a way to locate relevant orders without examining the entire history. The application must implement at least one option supported by the data, such as newest first, oldest first, status, date range, or order-number search.

11 Order Details

Customers need to select an order and review its available details. The screen should present customer-relevant information such as order number, date, status, shipping information, products, quantities, line totals, and order total when supplied by the API.

- The UI should omit internal fields that have no customer meaning.
- Unknown or missing optional values should be handled without displaying misleading information.

12 Order Status Explanation

Technical order-status values must be translated into understandable customer language. The application must handle an unfamiliar status without crashing or misrepresenting it.

- A reasonable fallback such as Unknown Status must be available.
- Status must not be communicated through color alone.

Required Application Features Continued

13 Refresh and Retry

Customers need a way to request current product and order information. Refresh operations must communicate progress, prevent uncontrolled duplicate requests, preserve existing content when reasonable, and provide recovery actions after failure.

14 Independent Section Failures

A failure retrieving one category of home-screen information must not unnecessarily hide unrelated information.

- Recently viewed products may remain available when order retrieval fails.
- Recent-order information may remain available when a product refresh fails.
- Saved products and browsing history should remain accessible during a network outage.

15 Complete UI State Handling

The application must deliberately represent and render the states that customers can encounter.

- Initial loading
- Successful data
- Empty catalog
- No search matches
- No saved products
- No recently viewed products
- No order history
- Connectivity failure
- Server or HTTP failure
- Invalid response data
- Request timeout

A blank screen, permanent progress indicator, raw exception message, or silent failure does not satisfy this requirement.

16 Accessibility and Responsive Presentation

Core customer workflows must be accessible and remain usable on supported phone sizes in portrait and landscape orientations.

- Provide meaningful TalkBack labels for interactive controls and informative images.
- Maintain a logical reading and traversal order.
- Use adequate touch targets and support text scaling.
- Do not communicate important information through color alone.
- Avoid clipped or unusable layouts on wider screens and in landscape orientation.

Technical Requirements

The application must demonstrate the architecture and Android technologies covered in the bootcamp. Teams may make reasonable implementation decisions, but the following constraints are required unless the instructor approves an alternative.

Required Technology

Area	Required approach
Language and UI	Kotlin and Jetpack Compose using Material 3
Navigation	Navigation Compose with typed or otherwise well-defined routes
Presentation	ViewModels exposing immutable StateFlow UI state
Dependency injection	Hilt with constructor injection as the normal default
Networking	Retrofit and OkHttp
Serialization	kotlinx.serialization using separate network DTOs
Persistence	Room for saved and recently viewed product data
Testing	JUnit and coroutine-aware unit tests; Compose UI testing is optional

Layer Responsibilities

Layer	Responsibilities
Presentation	Compose screens, route composables, ViewModels, UI state, and user events
Domain	Domain models, repository contracts, use cases, and customer-facing business rules
Data	Repository implementations, Retrofit services, Room DAOs, DTOs, entities, and mapping

Architectural Constraints

- Composable functions must not call Retrofit services or Room DAOs directly.
- ViewModels must not construct repositories, API services, or databases directly.
- Network DTOs and Room entities must remain separate from domain models.
- Domain classes must not import Android UI, Retrofit, OkHttp, Room, or Compose APIs.
- Repositories must provide the data boundary used by use cases and ViewModels.
- Mutable state must remain private to the class that owns it.
- Dependencies must be visible through constructors or Hilt modules rather than hidden service locators.

Networking Data and Persistence

Networking and Serialization

- Use suspend Retrofit functions for one-shot HTTP operations.
- Configure forward-compatible decoding for additional server fields.
- Represent timestamps with an unambiguous ISO 8601 value and map them to an appropriate domain type.
- Verbose HTTP body logging, if used, must be limited to debug builds and must not expose credentials or sensitive data.

Coroutines and Failure Handling

- Launch screen-bound operations from viewModelScope.
- Do not block the main thread and do not wrap Retrofit suspend calls in Dispatchers.IO without a specific blocking operation that requires it.
- Use structured concurrency when loading independent home-screen sections in parallel.
- Apply a reasonable timeout to operations that should not wait indefinitely.

Local Persistence

Room must persist saved products and recently viewed history. At minimum, the application must retain the product identifiers and the timestamps needed to order recently viewed products. Teams may also cache a minimal product snapshot or catalog data if they want more useful offline behavior.

Local data	Minimum behavior
Saved products	Persist product IDs until the customer removes them
Recently viewed	Persist product ID and viewed time; replace duplicates; retain latest ten
Demo session	Keep the assigned customer context in one centralized abstraction
Order data	Remote-only is acceptable; order caching is not required

Data Consistency

Saved state and recently viewed history are independent. Refreshing the catalog or encountering a network failure must not erase either collection. If a locally referenced product can no longer be retrieved, the application should handle the condition deliberately rather than crashing.

Scrum and Team Deliverables

Each four-person team is responsible for both the product and the process used to produce it. The business requirements must be converted into a prioritized backlog before implementation begins.

Required Scrum Artifacts

- A sample of the prioritized product backlog containing team-written user stories
- Acceptance criteria for stories selected for development
- Initial estimates and evidence that estimates were revisited when understanding changed
- A team Definition of Done
- A visible representation of a task board showing work status
- Evidence of backlog refinement
- A sprint review and product demonstration
- A brief retrospective identifying successes, difficulties, and one improvement for future work

Collaboration Expectations

- Work should be divided into vertical feature slices rather than permanent UI, data, and testing silos.
- Every team member must make meaningful code contributions.
- Pull requests should receive peer review before integration when the project workflow permits.
- Team members should share ownership of architecture, quality, and the final demonstration.
- Merge conflicts and integration work are team responsibilities rather than the responsibility of one designated student.

Suggested Initial Work Split

Initial pair	Vertical feature slice
Pair A	Product catalog and product details
Pair B	Order history and order details
Mixed pairing after integration	Saved and recently viewed persistence
Mixed pairing after integration	Home screen, error handling, accessibility, and tests

Definition of Done Minimum

- Acceptance criteria are satisfied.
- The code compiles and the application runs from a clean checkout using documented setup steps.
- Loading, empty, failure, and success states have been considered.
- Relevant automated tests pass.
- The feature meets the agreed architecture, accessibility, and code-review expectations.
- No credentials, secrets, generated build output, or verbose production logs are committed.

Testing and Quality Requirements

Testing should focus on behavior that is important, error-prone, or difficult to verify reliably by visual inspection. Teams are not expected to test every property or framework call.

Required Automated Test Coverage

Area	Required behaviors
Catalog	Case-insensitive search; combined filtering and sorting
Saved products	A product can be saved and removed without affecting history
Recent history	No duplicates, revisit moves to first, latest ten retained, and clear does not remove saved products
Orders	Date ordering and unknown order status behavior
ViewModel state	At least one important loading-to-success path and one failure path

Manual Quality Review

- Verify the application on at least one Android emulator or device configuration used by the team.
- Exercise the application with the API unavailable or deliberately returning an error.
- Confirm that saved and recent data survive an application restart.
- Check portrait, landscape, text scaling, TalkBack labels, touch targets, and non-color status communication.
- Review Logcat for crashes, repeated requests, leaked sensitive values, and uncaught exceptions.

Repository Submission

- Complete source code in the assigned Git repository
- A README containing setup instructions, demo customer configuration, base URL configuration, and known limitations
- A summary of automated tests and how to run them
- The final product backlog and Scrum artifacts requested by the instructor
- No secrets, local machine paths, or environment-specific generated files

Enhancements and Final Demonstration

Optional Enhancements

After completing the required MVP, teams may select one or two enhancements. Required functionality and reliability take priority over incomplete optional features.

- Compare two products
- Offline product catalog
- Recently viewed product snapshots for improved offline display
- Adaptive list-detail layout on larger screens
- Debounced search
- Animated list and saved-state transitions
- Order-status timeline visualization
- Additional accessibility or Compose UI tests

Stretch Features

The following capabilities should be attempted only after the required MVP and tests are complete:

- Login, logout, and token refresh
- Shopping cart or order creation
- Background synchronization or order-status notifications
- Full offline synchronization

Final Demonstration

Each team will deliver a short product demonstration. Every team member should participate.

- Introduce the customer problem and the team's product decisions.
- Demonstrate the catalog, search or filtering, product details, saved products, and recently viewed behavior.
- Demonstrate order history and order details for the assigned demo customer.
- Show one network failure or partial-failure recovery path.
- Explain the architecture and one important tradeoff made by the team.
- Show one persistence behavior and one automated test.
- Identify one accessibility or responsive-layout decision.
- Summarize one lesson from the retrospective.

Evaluation Criteria

The project will be evaluated as a working product increment. A reliable and well-explained MVP is more valuable than a collection of unfinished enhancements.

Area	Weight	Evidence
Customer functionality	30%	Required journeys work and acceptance criteria are satisfied
Architecture and separation	20%	Clear presentation, domain, and data boundaries with appropriate DI
Networking and coroutines	15%	Correct Retrofit, serialization, cancellation, parallel work, timeout, and failure handling
Persistence	10%	Saved and recent data behave correctly across restarts
UI and accessibility	10%	Usable Compose interface, meaningful states, accessibility, and responsive behavior
Testing and code quality	10%	Useful automated tests, readable code, and maintainable implementation
Scrum and demonstration	5%	Backlog quality, collaboration evidence, clear demo, and retrospective

Submission Checklist

- The application builds and runs using the README instructions.
- The assigned demo customer is configured in one centralized location.
- All required product, saved, recent, and order workflows are demonstrable.
- The application handles loading, empty, failure, timeout, and unknown-value conditions.
- Saved and recently viewed data survive an application restart.
- Required automated tests pass.
- Accessibility and responsive-layout checks have been completed.
- The repository contains no credentials, secrets, or machine-specific generated files.
- The backlog, acceptance criteria, Definition of Done, and requested Scrum artifacts are complete.
- The team is prepared to explain its architecture, decisions, limitations, and next steps.

Project Completion Standard

The capstone is complete when the team can demonstrate a coherent customer experience, explain how data travels through the architecture, show that important failure and persistence behaviors are intentional, and provide evidence that the team worked from a refined backlog rather than simply assembling disconnected screens.

Assigned Application User Identity

Team	User	Name
Aza, Jonah, Tyler, Nathan	11711	Daniel Davis
Blair, Gryffin, Manny, Scott	15251	Noah Hall
Michael, Nicolas, Shawn, Solomon	18749	Olivia Stewart
Anton, Deven, James, Thomas	11501	Brandy Chandra
Daniela, Harris, John, Miles	11502	Jared Peterson